

Introduction to Neural Networks

Biology, the McCulloch–Pitts Neuron, and Linear Separability

Introduction to Deep Learning · Class 2

28 July

Outline

The Biological Neuron

From Neuron to Computing Unit

The McCulloch–Pitts Unit

Computing with Threshold Logic

Geometry: Linear Separability

The Biological Neuron

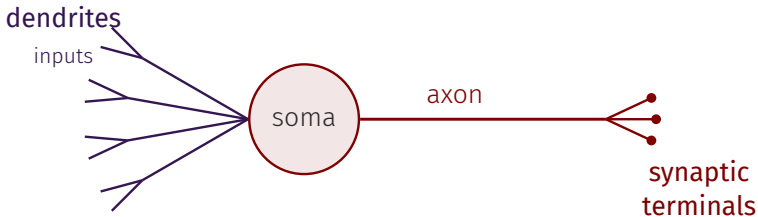
The brain as an inspiration

- The brain: roughly 10^{11} neurons, each connected to up to 10^4 others — massively **parallel**, not sequential
- Rojas: artificial neural networks are best understood as **networks of primitive functions** — simple units, wired together

Rojas (1996), §1.3

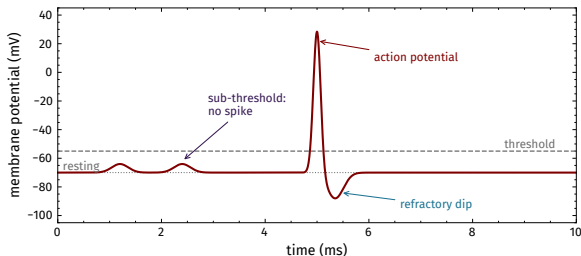
The plan: abstract the essential behaviour of a neuron into a simple computing unit, then study what networks of such units can do.

Anatomy of a neuron



- **Dendrites** collect incoming signals; the **soma** integrates them
- If the integrated signal crosses a threshold, the neuron **fires** an impulse down the **axon**
- **Synapses** pass the signal on — some excitatory, some inhibitory Rojas (1996), §1.2

Firing: an all-or-nothing threshold



Membrane potential over time. Small inputs raise it but decay away; once it crosses the threshold the neuron emits a sharp, stereotyped *action potential*, then dips below rest during the refractory period. The spike is **all-or-nothing**: its shape encodes no magnitude — only *whether* and *how often* the neuron fires.

BIO potential crosses a threshold → spike

MODEL weighted sum crosses a threshold → output 1

At the synapse: adding and subtracting

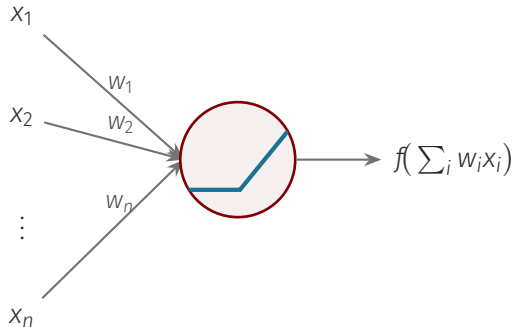
- Neurons meet at **synapses**; a spike releases chemical messengers that nudge the next neuron
- **Excitatory** synapses push the membrane *toward* threshold; **inhibitory** synapses push it *away*
- The soma continuously **sums** these pushes — thousands of them — and fires only if the net effect clears the threshold
- A single strong inhibitory input can **veto** many excitatory ones Rojas (1996), §1.2.3

BIO excitatory +, inhibitory −, integrated
at the soma

MODEL positive and negative **weights**,
summed in the unit

From Neuron to Computing Unit

The abstract neuron



- Each input channel i carries a value x_i , scaled by a **weight** w_i
- The body integrates the inputs (usually a sum) and applies a **primitive function** f

BIO dendrites $\times$ synaptic strengths,
summed at the soma

MODEL inputs $x_i \times$ weights w_i , summed in
the unit

Networks of primitive functions

- An artificial neural network can be visualized as a **network of primitive functions**

Rojas (1996), §1.3.1

- Models differ only in three choices:

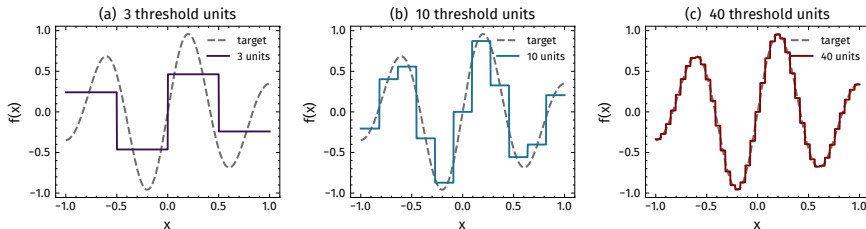
the **primitive**
functions used

the **interconnection**
pattern

the **timing** of
transmission

- Today's primitive function is the simplest possible — a **threshold**

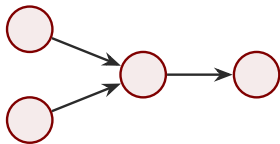
Simple units add up to complicated functions



Fitting $f(x) = \sin(2.4\pi x) e^{-x^2}$ (dashed) with a weighted sum of threshold units, each switching on at a different point. (a) 3 units, (b) 10, (c) 40 — the staircase converges on the target as units are added.

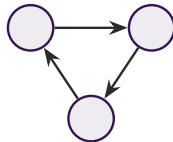
- Individually crude; **summed**, they approximate anything — this is a seed of universal function approximation

Feed-forward vs. recurrent



feed-forward

flows one way, no cycles



recurrent

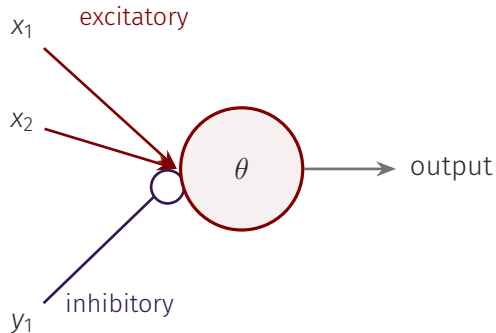
has cycles: memory

- **Feed-forward**: a directed acyclic graph — output is a fixed function of the input
- **Recurrent**: cycles let signals loop back, giving the network **state** and memory
- Focuses on feed-forward networks

Rojas (1996), §2.1.1

The McCulloch–Pitts Unit

The McCulloch–Pitts unit (1943)



- Inputs and output are **binary** (0 or 1); edges are unweighted
- Edges are **excitatory** or **inhibitory** (small circle)
- Each unit has a threshold θ Rojas (1996), §2.1.2

BIO excitatory / inhibitory synapses

MODEL excitatory / inhibitory edges into a threshold unit

The firing rule

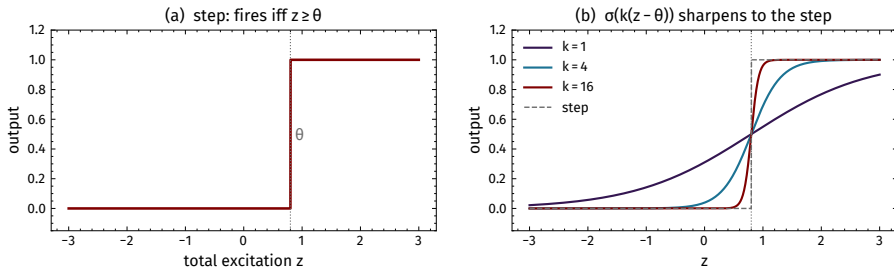
A unit receives $x_1, \dots, x_n$ on n excitatory edges and $y_1, \dots, y_m$ on m inhibitory edges

- If $m \geq 1$ and **any** inhibitory signal $y_j = 1$: the unit is **inhibited**, output = 0
- Otherwise compute $z = x_1 + \dots + x_n$ and compare with θ : output = 1 if $z \geq \theta$, else 0
- With no inhibition, the unit acts as a **threshold gate**

BIO fire iff input exceeds threshold; inhibition can veto

MODEL output 1 iff $z \geq \theta$; any active inhibitor forces 0

The activation is a step function

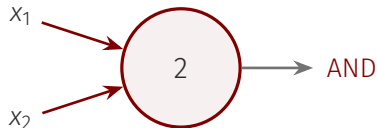


(a) the step (Heaviside) activation: the output jumps discontinuously from 0 to 1 at $z = \theta$. (b) a logistic $\sigma(k(z - \theta))$ for increasing sharpness k — as k grows it converges to the step.

- The hard threshold mirrors the neuron's all-or-nothing spike; the smooth versions in (b) return in Classes 5–6, where we need derivatives

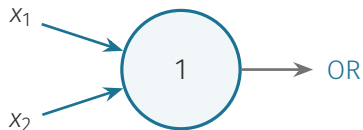
Computing with Threshold Logic

Boolean functions from threshold gates



Two binary inputs, 1 = *true*:

- **AND**: threshold $\theta = 2$ — fires only if *both* inputs are 1
- **OR**: threshold $\theta = 1$ — fires if *either* input is 1
- **NOT**: one inhibitory edge and $\theta = 0$

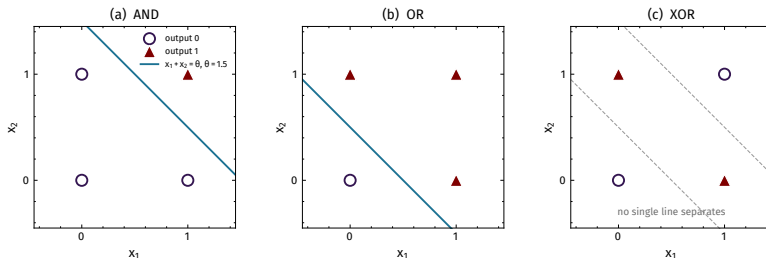


A single unit already computes the fundamental logic gates.

BIO a neuron as a coincidence detector

MODEL a threshold gate computing AND / OR

The same gates, drawn in input space



The four input pairs of a two-input Boolean function. Triangles output 1, open circles output 0. (a) AND and (b) OR are each split by the line $x_1 + x_2 = \theta$ — only θ differs. (c) XOR cannot be split by any straight line.

BIO a neuron responds to a region of stimulus space

MODEL a unit responds to a half-plane of inputs

Threshold logic is expressive

- A single unit can compute AND or OR of n arguments — just set $\theta = n$ (AND) or $\theta = 1$ (OR)
 - Negation needs an inhibitory edge: a unit with $\theta = 0$ and one inhibitory input computes NOT
 - With AND, OR, and NOT available, threshold-logic networks can compute any Boolean function
- Rojas (1996), §2.2

But how expressive is a *single* unit? The answer is geometric.

Any Boolean function, by construction

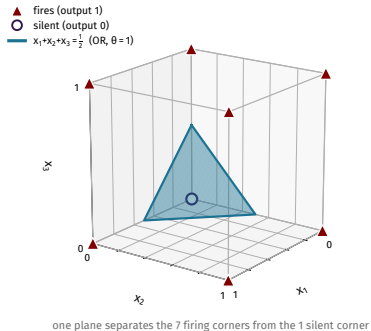
A logical function is a **table**: for each input vector, a 0 or a 1. To build a network for it (Rojas, 1996, §2.2.3):

1. For every input row that should output 1, use one unit as a **decoder** — it fires *only* for that exact vector
2. A decoder for $(1, 0, 1)$: excitatory edges from the 1-bits, inhibitory from the 0-bits, threshold = number of 1-bits
3. Feed all decoders into a single **OR** unit

Two layers of threshold units suffice for *any* Boolean function — expressive power is not the problem.

Geometry: Linear Separability

A unit splits input space into half-spaces



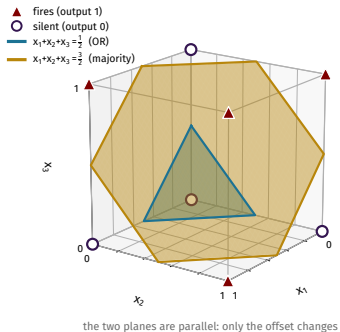
For three binary inputs and threshold θ , the unit tests $x_1 + x_2 + x_3 \geq \theta$:

- The equality $x_1 + x_2 + x_3 = \theta$ is a **separating plane**
- One side fires (▲), the other does not (○)
- OR: only the all-zero vertex lies below the plane

BIO a neuron responds to a region of stimulus space

MODEL a unit responds to a *half-space* of inputs

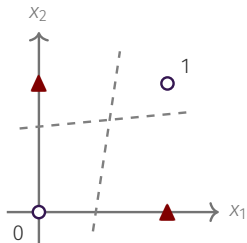
Parallel planes, different thresholds



- **OR**: plane at $\theta = 1$ (teal) — fires unless all inputs are 0
- **Majority**: plane at $\theta = 2$ (amber) — fires when at least two inputs are 1
- With unit weights the planes stay **parallel**
- **Non-parallel** planes need **weighted** edges — the perceptron (Class 3)

Vertices show the majority function.

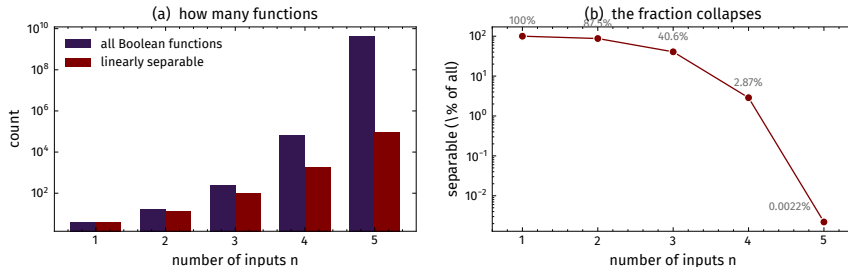
The one it cannot do: XOR



▲: XOR = 1; ○: XOR = 0.

- XOR outputs 1 when the inputs **differ**, 0 when they agree
- The two ▲ sit on **opposite corners** — so do the two ○
- **No single straight line** can put both ▲ on one side and both ○ on the other
- XOR is **not linearly separable** — a wall a single unit cannot climb

How rare is linear separability?



(a) of all 2^{2^n} Boolean functions of n inputs, only a small number are linearly separable (4, 14, 104, 1882, 94572 for $n = 1 \dots 5$). (b) as a percentage, that share collapses — by $n = 5$ it is about 0.002%.

- XOR is not a curiosity: **almost every** Boolean function is out of reach for a single unit

Linear separability — and its limit

- A single threshold unit computes exactly the **linearly separable** functions: those whose 1- and 0-inputs can be split by a hyperplane
- AND, OR, majority — all linearly separable
- XOR is not — and as n grows, almost nothing is

A single unit is a linear classifier. Escaping linear separability needs **weights** (Class 3) and, ultimately, **layers** (Classes 5–7).

Takeaways

1. A neuron integrates weighted inputs and fires when a **threshold** is crossed — all-or-nothing
2. The **McCulloch–Pitts unit** keeps only this: binary inputs, a threshold, excitatory/inhibitory edges
3. A single unit computes AND, OR, majority — but only **linearly separable** functions
4. Geometrically it is a **separating hyperplane**; XOR already escapes it, and almost every function does as n grows

Next Topic: add **weights** and a **learning rule** — the perceptron.

References

- R. Rojas. *Neural Networks: A Systematic Introduction*. Springer, 1996. Chapters 1–2 — biological motivation, threshold logic, geometric interpretation.
- W. S. McCulloch & W. Pitts (1943). “A logical calculus of the ideas immanent in nervous activity.” *Bulletin of Mathematical Biophysics*, 5, 115–133.
- Counts of linearly separable Boolean functions: OEIS A000609.